ABSTRACT

This report documents the work undertaken during a three-month internship at a technology company, wherein the primary project involved the consolidation of a two-tier server backend architecture into a single unified server, along with a comprehensive migration of the underlying technology stack.

The original system employed a two-server architecture to handle different layers of the backend, introducing operational complexity, inter-service communication overhead, and maintenance challenges. The objective of the internship project was to redesign and consolidate this architecture into a single-server model, eliminating redundancy while preserving all existing functionality and improving overall performance and maintainability.

Concurrently, the project required a full migration of the technology stack from the incumbent set of frameworks and libraries to a modern, more efficient alternative chosen by the engineering team. This migration involved rearchitecting request-handling pipelines, rewriting middleware layers, porting database interaction logic, and ensuring that all API contracts remained intact for downstream consumers.

The report covers the problem background and motivation, the theoretical foundations of the architectural decisions, a detailed description of the implementation methodology, challenges encountered and solutions devised, testing and validation strategies, outcomes and results, and personal learning from the internship experience.

The project was completed successfully within the internship duration, resulting in a simplified, consolidated backend system that demonstrated measurable improvements in response time, resource utilization, and code maintainability.

TABLE OF CONTENTS

UNDERTAKING……..i
CERTIFICATE…….....ii
ACKNOWLEDGEMENT……..…iii
ABSTRACT……iv

1. INTRODUCTION
1.1. Background……7
1.2. Motivation……..7
1.3. Problem Statement……7
1.4. Objectives……...8

2. EXISTING SYSTEM ANALYSIS
2.1. Original Two-Server Architecture..……9
2.2. Tech Stack Overview (Pre-Migration)……...…9
2.3. Identified Problems and Limitations……..9

3. PROPOSED SOLUTION
3.1. Single Server Architecture…….……..11
3.2. Tech Stack Selection (Post-Migration)……..…..11
3.3. Design Principles and Constraints……....11

4. IMPLEMENTATION
4.1. Server Consolidation…….……...13
4.2. Technology Stack Migration……13
4.3. Database Layer Integration……..……14
4.4. Middleware and Request Pipeline……14
4.5. Error Handling and Logging……15

5. TESTING AND VALIDATION
5.1. Unit Testing……..16
5.2. Integration Testing……...16
5.3. Performance Benchmarking……16

6. RESULTS AND OUTCOMES
6.1. Architectural Improvements…….……18
6.2. Performance Metrics……18
6.3. Code Quality and Maintainability……...……19

7. CHALLENGES AND SOLUTIONS
7.1. Underlying Existing Codebase…….……20
7.2. Ensuring API Contract Fidelity……20
7.3. Tech Stack Learning Curve……..……...……20

7.4. Transaction and State Management……20
7.5. Debugging in a Production Environment……21

8. LEARNING AND OUTCOMES
8.1. Technical Learnings…….……22
8.2. Professional and Collaborative Learnings……22

9. CONCLUSION
9.1. Conclusion……...…….……24

CHAPTER 1: INTRODUCTION

1.1 Background

The rapid evolution of backend software engineering has made architectural simplicity and performance optimization two of the most sought-after qualities in modern system design. Organizations increasingly seek to reduce the complexity of their infrastructure, lower operational costs, and improve developer productivity by streamlining the number of moving parts in their systems.

Server consolidation — the process of reducing multiple server nodes into fewer, more capable ones — is a well-established practice in both physical infrastructure and software architecture. In the context of software backend systems, this often means merging previously separated services or server layers into a single cohesive application, reducing inter-process communication, simplifying deployment pipelines, and unifying the development experience.

Similarly, technology stack migration is a common endeavour for organizations seeking to stay current with industry best practices, improve developer experience, or leverage performance gains offered by newer frameworks and tools. Such migrations require careful planning, meticulous execution, and thorough testing to ensure that existing functionality is fully preserved and no regressions are introduced.

This report documents an internship project that combined both of these goals: consolidating a two-tier backend server architecture into a single server while simultaneously migrating to a new technology stack — all within a real-world, production-oriented REST API backend service.

1.2 Motivation

The motivation for undertaking this project stemmed from a combination of practical operational pain points observed within the organization and strategic technical goals set by the engineering leadership team. From an operational standpoint, the existing two-server architecture had introduced measurable friction in day-to-day engineering activities. Developers working on features that touched both server layers had to context-switch between two separate codebases, manage two sets of dependencies, and coordinate changes across two deployment pipelines. This fragmentation slowed down development cycles and increased the surface area for bugs, particularly those that arose from subtle mismatches in the internal API contracts between the two servers.

From a performance perspective, the inter-server HTTP communication introduced by the two-server model added latency to every single client request. Since Server A had to make a network call to Server B for virtually every operation, the overhead of this internal round trip

was a structural tax on the system's responsiveness. As traffic volumes grew, this latency contribution became increasingly significant.

1.3 Problem Statement

The existing architecture, while functionally correct, introduced several practical challenges that impacted performance, maintainability, and development efficiency. Each client-facing API request experienced added latency because it required an additional internal HTTP round trip between the two servers, increasing network overhead regardless of the request's complexity. From an operational standpoint, maintaining two separate server processes led to duplication of infrastructure efforts, including deployment pipelines, environment configurations, monitoring systems, and logging setups, thereby increasing maintenance overhead. Development workflows were also affected, as implementing new features or fixing bugs often required coordinated changes across two distinct codebases, which not only slowed down development but also increased the likelihood of integration issues. Furthermore, error handling became more complex, since errors originating in one server had to be serialized, transmitted, and reinterpreted by the other, introducing the risk of information loss or inconsistent error responses. On top of these issues, the legacy technology stack used by both servers had accumulated significant technical debt and no longer aligned with the team's performance expectations and productivity standards, ultimately necessitating a migration to a more modern and efficient stack.

1.4 Objectives

The internship project was guided by a set of clearly defined objectives that structured the entire workflow. It began with a comprehensive analysis of the existing two-server architecture, including a detailed understanding of each server's responsibilities, inter-server communication, database interactions, and all exposed REST API endpoints. Based on this, a new consolidated single-server architecture was designed, ensuring proper separation of concerns through layers such as routing, middleware, controllers, services, and data access, while eliminating the need for inter-server network calls. The next step involved implementing this design by merging the functionalities of both Server A and Server B into a unified codebase, while preserving existing API contracts and business logic. Alongside this, a complete migration to a modern technology stack was carried out, adhering to updated best practices and improving overall developer productivity. The database layer was also seamlessly integrated into the new architecture, incorporating efficient connection handling, transaction management, and structured data access patterns. To ensure reliability, a thorough testing strategy was implemented, including unit and integration tests, along with performance benchmarking to validate improvements. Finally, the project was documented in detail, covering architectural decisions, implementation strategies, challenges faced, and the overall outcomes of the internship.

CHAPTER 2: EXISTING SYSTEM ANALYSIS

2.1 Original Two-Server Architecture

The existing system was built upon a two-tier server architecture. In this design, the backend consisted of two distinct server processes, each operating independently and communicating with each other over the network. The first server — referred to hereafter as Server A — acted as the primary entry point for all incoming API requests. It was responsible for authentication and authorization, request validation, routing logic, and orchestrating calls to the second server. Server B, the secondary server, handled the core business logic and was the only layer that directly communicated with the database. It received processed requests from Server A, performed the necessary data operations, and returned structured responses. This design created a clear separation of concerns between the API gateway layer and the business logic layer.
The communication between Server A and Server B occurred over HTTP, using internally defined API contracts. This meant that every client request resulted in at least two HTTP round trips within the backend infrastructure itself, before any response could be formulated — one from the client to Server A, and one from Server A to Server B.

2.2 Tech Stack Overview (Pre-Migration)

In accordance with the NDA obligations, specific technology names from the original stack cannot be disclosed in this report. In general terms, the existing stack was composed of a set of frameworks and libraries for server-side request handling, middleware processing, data validation, and database interaction. Each server maintained its own dependencies, configuration files, environment variables, and deployment scripts, leading to duplicated infrastructure and separate maintenance overhead.

The original stack, while functional, had accumulated technical debt over time and was identified by the engineering team as being less aligned with modern backend development paradigms. This formed one of the primary motivations for the planned technology stack migration.

2.3 Identified Problems and Limitations

A detailed analysis of the existing architecture, carried out in collaboration with the engineering team during the initial phase of the internship, uncovered several key limitations. One of the primary issues was the added network latency, as every client request required an extra internal HTTP round trip between Server A and Server B, which significantly impacted response times, especially under high load. The architecture also introduced operational complexity, as maintaining two separate server processes meant duplicating efforts in deployment, monitoring, logging, and alerting, with any infrastructure change needing to be applied to both servers. Error handling was another challenge, as errors generated in Server B had to be serialized, transmitted, and reformatted by Server A before being returned to the client,

sometimes resulting in inconsistencies or loss of detail. From a development perspective, implementing features that spanned both servers required coordinated updates across two codebases, slowing down development and increasing the risk of mismatched API contracts. Additionally, running two servers led to unnecessary resource consumption, even during low-traffic periods. Finally, the existing technology stack posed limitations, as it did not align well with the team's long-term goals for performance, ecosystem support, and developer productivity.

CHAPTER 3: PROPOSED SOLUTION

3.1 Single Server Architecture

The proposed solution was to consolidate both Server A and Server B into a single unified server application. This consolidated server would assume all responsibilities previously divided between the two servers: request intake, authentication and authorization, validation, business logic execution, and direct database communication.

The architecture of the consolidated server was organized into a layered internal structure, clearly delineating the different functional concerns as separate modules within the same application. This preserved the conceptual separation of concerns of the original two-server design, but eliminated the network boundary between them. The key layers in the consolidated server were: the routing layer, the middleware layer, the controller layer, the service layer, and the data access layer (DAL).

By removing the inter-server network hop, every client request would now be handled entirely within a single process, drastically reducing response latency and eliminating the complexity associated with inter-service communication.

3.2 Tech Stack Selection (Post-Migration)

The new technology stack was selected by the engineering leadership team prior to the commencement of the internship. The selection criteria centered around performance characteristics, ecosystem maturity, quality of documentation, developer productivity features, and alignment with the long-term technical roadmap of the organization.

In accordance with the NDA, specific names of the new technology stack cannot be disclosed. In general terms, the new stack comprised a modern, high-performance server framework, an updated approach to middleware composition, improved tooling for database interaction, and a more expressive way to define and enforce data validation schemas.

The new stack was designed to offer better raw throughput compared to the legacy stack, a more predictable and type-safe development experience, improved observability primitives (logging, tracing), and a reduced footprint in terms of third-party dependencies.

3.3 Design Principles and Constraints

Several guiding principles governed the design and execution of the proposed solution:

- API Contract Preservation: All existing REST API endpoints, request/response schemas, and HTTP status code behaviour had to remain identical to ensure backward compatibility with existing clients and consumers of the API.

- Zero Functional Regression: Every piece of business logic present in the original two-server system had to be faithfully ported to the consolidated server. No feature could be dropped or altered in behaviour.
- Modularity: The consolidated server was to be designed in a modular fashion, ensuring that individual components (routing, middleware, business logic, DAL) could be independently understood, tested, and modified.
- Testability: The new codebase had to be structured to facilitate unit and integration testing at every layer.
- Incremental Migration: Given the production-critical nature of the service, the migration was to be performed incrementally where possible, with continuous testing after each phase.
- Documentation: All architectural decisions and implementation approaches were to be documented for future reference by the engineering team.

CHAPTER 4: IMPLEMENTATION

4.1 Server Consolidation

The consolidation process was undertaken systematically, starting with a comprehensive mapping exercise. Every API endpoint exposed by Server A was catalogued, along with its corresponding handler in Server B. This produced a complete inventory of all routes, their expected request formats, the business logic they invoked, and the database operations they ultimately triggered.

Using this inventory, each endpoint was migrated one at a time into the consolidated server. The process for each endpoint followed a consistent pattern: first, the route definition was created in the routes layer; second, the corresponding controller was implemented; third, the business logic was ported into a service function; and fourth, the database access code was encapsulated in a repository function.

Special attention was paid to endpoints that involved multi-step operations within Server B, such as those requiring database transactions, conditional logic based on query results, or chained database calls. These were ported carefully to ensure that the transactional integrity and logical flow were preserved identically.

Authentication and authorization logic, which was originally handled entirely within Server A, was ported into the middleware layer of the consolidated server, ensuring that it continued to protect all relevant routes before any business logic was executed.

4.2 Technology Stack Migration

The technology stack migration was executed in parallel with the server consolidation effort. While the high-level architecture of controllers, services, and repositories was common to both the old and new designs, the specific syntax, APIs, and idioms of the new stack required every piece of code to be rewritten rather than simply copied over.

The migration involved the following key activities:

- Server Initialization: The entry point of the application was rewritten using the new framework's server initialization API. This included configuring the HTTP server, binding middleware, and registering the router.

- Routing: Route definitions were rewritten using the new stack's routing primitives. Care was taken to ensure that HTTP methods, URL patterns (including parameterized routes and query parameters), and response formats exactly matched those of the original system.

- Middleware: Each piece of middleware from both original servers was analysed, understood, and reimplemented using the new stack's middleware composition pattern.

This included authentication token parsing and verification, request body parsing, input validation, logging, and error catching.

- Business Logic: Service functions were carefully rewritten, preserving all conditional logic, data transformations, and computation present in the original code. Where the original logic was unclear, the engineering mentor was consulted to clarify the intended behaviour.

- Configuration Management: Environment variable handling and application configuration were migrated to the new stack's recommended configuration management approach.

4.3 Database Layer Integration

In the original two-server design, all database access was confined to Server B. With the consolidation, the database layer was now directly integrated into the single consolidated server, specifically within the repository layer.

The database connection was configured using the new stack's database client or ORM, with connection pooling properly configured to handle concurrent request loads efficiently. The connection lifecycle was managed centrally in the application startup and shutdown hooks, ensuring that connections were cleanly established on startup and gracefully closed on shutdown.

Each repository module was responsible for a specific domain entity (e.g., user repository, order repository) and exposed a clear set of data access functions. All SQL queries or ORM method calls were encapsulated within these repository functions, ensuring that no direct database calls existed elsewhere in the codebase. This separation facilitated easier testing through mocking and simplified future database schema changes.

Database transactions, where required for atomicity, were implemented using the new stack's transaction management primitives, ensuring that operations that needed to succeed or fail together were properly wrapped in transactional contexts.

4.4 Middleware and Request Pipeline

One of the architectural improvements introduced in the consolidated server was a cleaner and more explicit request processing pipeline. In the original system, the middleware chain was split across two servers, making it difficult to trace the full lifecycle of a request. In the consolidated server, the entire pipeline from request receipt to response dispatch was visible and configurable within a single application.

The request pipeline was organized as follows: incoming requests first passed through global middleware (logging, body parsing, CORS handling), then through route-specific middleware

(authentication guards, role-based authorization, input validation), then to the controller, which invoked service functions and repository calls, and finally produced a structured response returned to the client.

Input validation middleware was implemented using the new stack's schema validation capabilities, ensuring that requests with invalid payloads were rejected at the middleware layer before reaching any business logic, with descriptive and consistent error messages returned to the client.

4.5 Error Handling and Logging

A centralized error handling mechanism was implemented as a global error-catching middleware in the consolidated server. This middleware intercepted all errors thrown within the request pipeline, classified them by type (validation errors, authentication errors, not-found errors, internal server errors), and returned appropriately formatted JSON error responses with the correct HTTP status codes.

This was a significant improvement over the original system, where errors from Server B had to be serialized and re-communicated to Server A before reaching the client, sometimes resulting in generic or misleading error messages. In the consolidated server, errors were handled in a single place with full context, producing more accurate and informative error responses.

A structured logging system was integrated into the server, logging all incoming requests, outgoing responses, and internal errors with consistent metadata including request identifiers, timestamps, HTTP method and path, response status codes, and processing times. This structured logging greatly improved the observability of the service compared to the original system.

CHAPTER 5: TESTING AND VALIDATION

5.1 Unit Testing

Unit tests were written for all service functions and repository functions in the consolidated server. The testing strategy focused on isolating each function from its dependencies through mocking — database calls in service tests were mocked to return predefined data, and external service calls (where applicable) were similarly mocked.

Each unit test covered the primary happy-path scenario, edge cases (empty inputs, boundary values, null cases), and expected error scenarios (invalid inputs, not-found cases, unauthorized access). Writing these tests often helped identify subtle bugs in the migrated business logic, particularly in cases where the original code had implicit assumptions that needed to be made explicit in the new implementation.

The unit test suite achieved a high level of coverage across the service and repository layers, providing confidence that the core business logic was correctly ported from the original system.

5.2 Integration Testing

Integration tests were designed to test the full request-response lifecycle of each API endpoint. These tests made actual HTTP requests to a running instance of the consolidated server connected to a dedicated test database, and verified that the response status codes, headers, and body payloads matched the expected values.

For each endpoint, integration tests covered: successful request scenarios with valid inputs, request scenarios with various categories of invalid inputs (missing required fields, incorrect data types, out-of-range values), authentication scenarios (valid token, missing token, expired token, insufficient permissions), and resource scenarios (existing resource, non-existent resource, duplicate resource creation).

The integration test suite served as a regression test suite throughout the development process. Every time a new endpoint was migrated to the consolidated server, its integration tests were run to verify correctness before proceeding to the next endpoint.

5.3 Performance Benching

A performance benchmarking exercise was conducted to compare the response times of the original two-server system with the consolidated single-server implementation. Benchmarks were run using a load testing tool configured to simulate concurrent client requests across several representative API endpoints.

The benchmarking results confirmed the expected performance improvement resulting from the elimination of the inter-server HTTP round trip. Under simulated load, the consolidated server demonstrated meaningfully lower average response times and higher request throughput compared to the original architecture. Details of specific numbers are not disclosed in this report due to their commercially sensitive nature.

CHAPTER 6: RESULTS AND OUTCOMES

6.1 Architectural Improvements

The primary outcome of the project was the successful consolidation of the two-server backend into a single unified server. The resulting architecture was significantly simpler in terms of deployment topology, operational surface area, and inter-component communication. The removal of the internal HTTP layer between what were previously Server A and Server B resulted in a more cohesive and easier-to-reason-about system.

The new architecture also introduced a much more explicit and traceable request processing pipeline, improved centralized error handling, and a clearly defined layered structure that improved the navigability of the codebase for developers.

6.2 Performance Metrics

The performance improvements observed during benchmarking are summarized qualitatively below. Specific numeric values are withheld per NDA obligations.

Table 6.1: Comparative performance and quality metrics between the original and consolidated system.

Metric	**Original System**	**Consolidated System**	**Trend**
Avg. API Response Time	Baseline	Significantly Lower	Improved
Request Throughput (RPS)	Baseline	Notably Higher	Improved
Server Resource Usage	2 Processes	1 Process	Reduced
Deployment Complexity	High (2 servers)	Low (1 server)	Reduced

Error Traceability	Moderate	High	Improved
Test Coverage	Low	High	Improved

6.3 Code Quality and Maintainability

The migration to the new technology stack, combined with the architectural consolidation, produced a codebase that was significantly easier to read, understand, and maintain. The new stack's idioms encouraged cleaner separation of concerns and more predictable data flows. The structured layered architecture (routes, controllers, services, repositories) made it straightforward for any developer to locate and modify specific pieces of functionality without needing to understand the entire system.

The introduction of comprehensive unit and integration test suites also markedly improved the confidence with which future changes could be made, reducing the risk of regressions and providing a safety net for ongoing development.

CHAPTER 7: CHALLENGES AND SOLUTIONS

7.1 Underlying Existing Codebase

One of the earliest and most significant challenges was developing a thorough understanding of the existing two-server codebase. The original code had been developed over an extended period by multiple engineers and contained areas of undocumented business logic, implicit assumptions, and tightly coupled components. Without a full understanding of these, migrating the code accurately would be impossible.

This challenge was addressed through a combination of systematic code reading, leveraging the integration test suite (where it existed), conducting focused sessions with the engineering mentor to clarify ambiguous logic, and building a comprehensive endpoint inventory document that captured the behaviour of every API.

7.2 Ensuring API Contract Fidelity

A critical constraint of the project was that all existing API contracts — endpoint URLs, HTTP methods, request body shapes, query parameter formats, response body schemas, and HTTP status codes — had to be preserved exactly. Any deviation would constitute a breaking change for the clients of the API.

This was managed by treating the integration test suite as the ground truth for API behavior. Tests were written against the original system first, capturing expected responses for a wide range of input scenarios, and then run against the consolidated server to verify that behavior was identical.

7.3 Tech Stack Learning Curve

The new technology stack, while modern and well-documented, was new to me at the commencement of the internship. Ramping up on its idioms, APIs, and best practices while simultaneously working on a production-oriented project presented a significant challenge, particularly in the early weeks.

This was addressed through dedicated self-study time allocated by the engineering team in the first two weeks of the internship, supplemented by code review feedback from the engineering mentor, consultation of official documentation, and building small prototype implementations to validate understanding before applying patterns to the main project.

7.4 Transaction and State Management

Certain API operations in the original system involved multi-step database transactions — sequences of database operations that needed to succeed atomically or be rolled back entirely. Translating these correctly into the new stack's transaction management primitives required

careful analysis and testing, particularly for complex operations involving multiple tables or conditional logic mid-transaction.

Each transactional operation was tested with specific integration test cases that verified both the success path and rollback behaviour (by simulating failures at various stages), ensuring that the transactional semantics were correctly preserved.

7.5 Debugging in a Production Environment

The testing environment used during the internship closely mirrored the production configuration, which meant that debugging issues required understanding the interaction of the server with real database states and realistic data volumes. Some bugs only manifested under specific data conditions that were not captured in the initial test cases.

This was addressed by expanding the test data sets used in integration testing and by using structured logging to capture detailed context around failures, which significantly accelerated the diagnosis and resolution of such issues.

CHAPTER 8: LEARNING OUTCOMES

8.1 Technical Learnings

Backend Architecture Design: The internship provided deep hands-on experience with the principles and trade-offs of server architecture design. Understanding the real-world implications of the two-server model — its operational complexity, latency overhead, and development friction — and then designing and implementing the single-server alternative gave me a thorough, practical understanding of concepts that are often taught only theoretically.

REST API Development: Working on a production REST API service solidified my understanding of RESTful design principles, HTTP semantics, status code conventions, versioning strategies, and the importance of consistent API contract design.

Middleware and Request Pipelines: Implementing a full middleware chain — from authentication guards and input validation to error handling and structured logging — gave me a detailed understanding of how modern web servers process requests from receipt to response.

Database Interaction Patterns: Implementing the repository pattern for database access, managing connection pools, and handling database transactions in a production-oriented context deepened my understanding of data access layer design and the importance of separating persistence concerns from business logic.

Software Testing: Writing comprehensive unit and integration test suites as a first-class concern — not as an afterthought — fundamentally changed my appreciation for automated testing. Experiencing the value of a good test suite as a safety net during a large-scale migration was a powerful and lasting lesson.

Technology Migration: Successfully executing a full technology stack migration in a production-oriented codebase taught me the importance of incremental migration, maintaining backward compatibility, and using tests as the primary mechanism for verifying correctness during transitions.

8.2 Professional and Collaborative Learnings

Agile Development Practices: Participating in sprint planning, daily standups, sprint retrospectives, and backlog grooming gave me practical experience with agile methodologies as they are actually practiced in professional software teams.

Code Review Culture: Submitting code for review and receiving constructive feedback from senior engineers was an invaluable learning experience. It exposed me to alternative approaches, best practices I had not previously encountered, and a higher standard of code quality than I had previously applied in academic contexts.

Technical Communication: Writing design documents, maintaining an endpoint inventory, and documenting architectural decisions honed my ability to communicate technical concepts clearly and concisely in written form — a skill that is critical in professional software engineering.

Problem Decomposition: Working on a large, complex project taught me the importance of breaking down a big problem into smaller, well-defined sub-tasks, prioritizing them logically, and executing them incrementally with frequent validation checkpoints.

CHAPTER 9: CONCLUSION

This internship project successfully achieved its primary objectives: the consolidation of a two-tier backend server architecture into a single unified server, and the migration of the underlying technology stack, for a production-oriented REST API backend service.

The consolidated system demonstrated clear improvements over its predecessor across multiple dimensions — lower API response latency, higher request throughput, reduced operational complexity, improved error handling and observability, and significantly better code maintainability. The comprehensive test suite developed during the project provides the engineering team with a strong safety net for ongoing development and future changes.

The internship was an immersive and transformative professional experience. It bridged the gap between academic learning and real-world software engineering in a way that no coursework alone could replicate. The technical challenges encountered and overcome during the project — understanding legacy codebases, preserving API contracts across a migration, managing transactions, and debugging in realistic environments — have equipped me with skills and perspectives that will be directly applicable throughout my engineering career.